

Reviewer Pack — Slice Rank Lower Bound for Disjointness Communication Matric...

Entry ea46e1d39086 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 22:15:04 UTC

Slice Rank Lower Bound for Disjointness Communication Matrices

Entry ID: ea46e1d39086 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-09 22:15:04 UTC

1. Conjecture

Field A (mathematical branch): Additive Combinatorics (Slice Rank) **Field B** (complexity object): Communication Complexity of DISJOINTNESS

Statement:

For the $n \times n$ matrix M representing the DISJOINTNESS problem, the slice rank of M is $\Omega(n)$, implying that the randomized communication complexity of DISJOINTNESS is $\Omega(n)$.

Rationale (proposer's reasoning):

Slice rank, a measure from additive combinatorics, captures the minimal decomposition of a tensor into rank-1 components. By linking it to communication complexity, we may uncover structural constraints on the problem's information-theoretic requirements.

Taxonomy category: COMM_DISJ (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 12cf62628464a8c0

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -slice rank disjointness communication complexity -slice rank randomized communication complexity disjointness -slice rank lower bound disjointness communication matrix

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import combinations

def generate_disjointness_matrix(n):
    matrix = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(i + 1, n):
            matrix[i][j] = 1
            matrix[j][i] = 1
    return matrix

def tensor_product(t1, t2):
    result = []
    for a in t1:
        row = []
        for b in t2:
            row.extend([x * y for x, y in zip(a, b)])
        result.append(row)
    return result

def rank_1_decomposition(tensor):
    n = len(tensor)
    U = [[0] * n for _ in range(n)]
    V = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            if tensor[i][j] != 0:
                U[i][i] = 1
                V[j][j] = 1
                break
    return U, V

def slice_rank(tensor):
    n = len(tensor)
    rank_1_count = 0
    for i in range(n):
        for j in range(n):
            sub_tensor = [row[j:] for row in tensor[i:]]
            u, v = rank_1_decomposition(sub_tensor)
            if all(all(x == 0 or x == 1 for x in row) for row in u) and all(all(x == 0 or x == 1 for x in row) for row in v):
                rank_1_count += 1
```

```

        rank_1_count += 1
    return rank_1_count

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []
    for n in n_values:
        matrix = generate_disjointness_matrix(n)
        rank = slice_rank(matrix)
        results.append({
            "metric_name": "slice_rank",
            "metric_value": rank,
            "instances_tested": 1,
            "conjecture_holds": rank >= n,
            "counterexample": "" if rank >= n else f"n={n}, rank={rank}"
        })
    return {
        "seed": seed,
        "results": results
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.extend(result["results"])

    if not results:
        print("RESULT: INCONCLUSIVE no trials executed")
    else:
        mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
        support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)
        if support_fraction >= 0.8:
            print(f"RESULT: SUPPORTED mean={mean_metric_value:.2f} std=0.00 support_fraction={support_fraction:.2f}")
        else:
            first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
            counterexample = next(result["counterexample"] for result in results if not result["conjecture_holds"])
            print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3e07a911.py", line 81, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3e07a911.py", line 63, in run_trial

```

```

rank = slice_rank(matrix)
^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3e07a911.py", line 52, in slice_rank
    u, v = rank_1_decomposition(sub_tensor)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3e07a911.py", line 40, in rank_1_decomp
    if tensor[i][j] != 0:
    ~~~~~~^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with IndexError, preventing reliable results. The crash indicates potential issues in tensor indexing logic that need resolution before further analysis. | next: Fix tensor indexing bounds in rank_1_decomposition and re-validate with controlled test parameters

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	109592
2	propose	ollama_remote	qwen3:8b	0	0	101228
3	propose	ollama_remote	qwen3:8b	0	0	112140
4	propose	ollama_remote	qwen3:8b	0	0	91370
5	preregistration	ollama_remote	qwen3:8b	0	0	24206
6	novelty	ollama_remote	qwen3:8b	0	0	20534
7	novelty	ollama_remote	qwen3:8b	0	0	19213
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14051
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10338
10	judge	ollama_remote	qwen3:8b	0	0	17720

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 520393 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in [research/audit/ea46e1d39086.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/ea46e1d39086.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/ea46e1d39086.tar.gz` (if generated)